

6.4 Genetic Algorithm

A *genetic algorithm* is a heuristic search method used in artificial intelligence. It is used for finding optimized solutions to search problems based on the theory of natural selection and evolutionary biology. Genetic algorithms are excellent for searching through large and complex data sets. They are considered capable of finding reasonable solutions to complex issues as they are highly capable of solving unconstrained and constrained optimization issues.

A genetic algorithm makes use of techniques inspired from evolutionary biology such as selection, mutation, inheritance and recombination to solve a problem. The most commonly employed method in genetic algorithms is to create a group of individuals randomly from a given population. The individuals thus formed are evaluated with the help of the evaluation function provided by the programmer. Individuals are then provided with a score which indirectly highlights the fitness to the given situation. The best two individuals are then used to create one or more offspring, after which random mutations are done on the offspring. Depending on the needs of the application, the procedure continues until an acceptable solution is derived or until a certain number of generations have passed.

A genetic algorithm differs from a classical, derivative-based, optimization algorithm in two ways:

- A genetic algorithm generates a population of points in each iteration, whereas a classical algorithm generates a single point at each iteration.
- A genetic algorithm selects the next population by computation using random number generators, whereas a classical algorithm selects the next point by deterministic computation.

Compared to traditional artificial intelligence, a genetic algorithm provides many advantages. It is more robust and is

susceptible to breakdowns due to slight changes in inputs or due to the presence of noise. With respect to other optimization methods like linear programming, heuristic, first or breadth-first, a genetic algorithm can provide better and more significant results while searching large multi-modal state spaces, large state spaces or n -dimensional surfaces.

Genetic algorithms are widely used in many fields such as robotics, automotive design, optimized telecommunications routing, engineering design and computer-aided molecular design.

Criteria for efficient search algorithm for complex problems:

- randomly generate a population of potential solutions
- calculate fitness of each potential solution
- allow best individuals to breed and crossover ($p < 0.6$)
- allow low probability mutations ($p < 0.007$) to maintain diversity
- check for convergence of populations in the gene pool

Genetic Algorithm Components or Operators

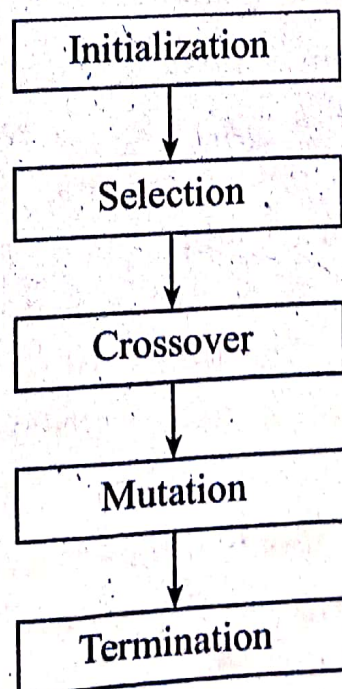


Figure 6.6: Genetic algorithm



1) Initialization

The population size depends on the nature of the problem, but typically contains several hundreds or thousands of possible solutions. Often, the initial population is generated randomly, allowing the entire range of possible solutions in the search space. Occasionally, the solutions may be "seeded" in areas where optimal solutions are likely to be found.

2) Selection

During each successive generation, a portion of the existing population is selected to breed a new generation. Individual solutions are selected through a fitness-based process. Certain selection methods rate the fitness of each solution and preferentially select the best solutions. Other methods rate only a random sample of the population, as the former process may be very time-consuming.

The fitness function is defined over the genetic representation and measures the quality of the represented solution. The fitness function is always problem dependent. For instance, in the knapsack problem, one wants to maximize the total value of objects that can be put in a knapsack of some fixed capacity. A representation of a solution might be an array of bits, where each bit represents a different object, and the value of the bit (0 or 1) represents whether or not the object is in the knapsack. Not every such representation is valid, as the size of objects may exceed the capacity of the knapsack. The fitness of the solution is the sum of values of all objects in the knapsack if the representation is valid, or 0 otherwise.

In some problems, it is hard or even impossible to define the fitness expression; in these cases, a simulation may be used to determine the fitness function value of a phenotype (e.g. computational fluid dynamics is used to determine the air resistance of a vehicle whose shape is encoded as the phenotype), or even interactive genetic algorithms are used.

3) Crossover and Mutation

The next step is to generate a second generation population of solutions from those selected through a combination of genetic operators: crossover (also called recombination), and mutation.

For each new solution to be produced, a pair of "parent" solutions is selected for breeding from the pool selected previously. By producing a "child" solution using the above methods of crossover and mutation, a new solution is created which typically shares many of the characteristics of its "parents". New parents are selected for each new child, and the process continues until a new population of solutions of appropriate size is generated. Although reproduction methods that are based on the use of two parents are more "biology inspired" i.e., generate higher quality chromosomes. It is done at random position and offspring produced have 30-70% of its parent's character. We randomly select a crossover point.

These processes ultimately result in the next generation population of chromosomes that is different from the initial generation. Generally, the average fitness will have increased by this procedure for the population, since only the best organisms from the first generation are selected for breeding, along with a small proportion of less fit solutions. These, less fit solutions ensure genetic diversity within the genetic pool of the parents and therefore ensure the genetic diversity of the subsequent generation of children by mutation. In mutation we flip the value of the gene randomly. It is said that only 1% of data is mutated at single iteration. Mutation help to achieve the diversity in the solution.

4) Termination

This generational process is repeated until a termination condition has been reached. Common terminating conditions are:

- A solution is found that satisfies minimum criteria
- Fixed number of generations reached
- Allocated budget (computation time/money) reached

- The highest ranked solution fitness is reaching or has reached a plateau such that successive iterations no longer produce better results
- Manual inspection
- Combinations of the above

When to Use Genetic Algorithm?

Genetic algorithms are not good for all kinds of problems. They're best for problems where there is a clear way to evaluate fitness. If our search space is not well constrained or our evaluation process is computationally expensive, it may not find solutions in a same amount of time. Genetic algorithm is a decent algorithm in place, but the "knobs" just need to be tweaked.